SUBJECT CODE & NAME: CSA17 - ARTIFICIAL INTELLIGENCE

NAME:J.ARCHANA
REGISTER NUMBER:192324294

LAB RECORD - DAY 2

List of Programs

1. Map Coloring (CSP) *(CO5)*
2. Tic Tac Toe Game *(CO3)*
3. Minimax Algorithm *(CO3)*
4. Alpha-Beta Pruning *(CO3)*
5. Decision Tree *(CO4)*
6. Feed Forward Neural Network *(CO4)*
7. Sum of Integers from 1 to N (Prolog) *(CO4)*
8. Database with Name and DOB (Prolog) *(CO5)*
9. Student-Teacher-Subject Code (Prolog) *(CO1)*
10. Planets Database (Prolog) *(CO2)*

Program 1: Map Coloring (CSP)

Course Outcome: CO5

Aim: To write a Python program to colour the regions of a map so that no two adjacent regions share the same colour, by implementing a Constraint Satisfaction Problem (CSP) with backtracking.

Program:

```
regions = ["WA", "NT", "SA", "Q", "NSW", "V", "T"]
neighbors = {
    "WA": ["NT", "SA"],
    "NT": ["WA", "SA", "Q"],
    "SA": ["WA", "NT", "Q", "NSW", "V"],
    "Q": ["NT", "SA", "NSW"],
    "NSW": ["Q", "SA", "V"],
    "V": ["SA", "NSW"],
    "T": [],
}
colors = ["Red", "Green", "Blue"]


def is_consistent(region, color, assignment):
    for nb in neighbors[region]:
        if assignment.get(nb) == color:
            return False
    return True


def backtrack(assignment):
    if len(assignment) == len(regions):
        return assignment
    region = next(r for r in regions if r not in assignment)
    for color in colors:
        if is_consistent(region, color, assignment):
            assignment[region] = color
            result = backtrack(assignment)
            if result:
                return result
            del assignment[region]
    return None


print("MAP COLORING PROBLEM (CSP - Backtracking)")
print("Regions  :", regions)
print("Colors   :", colors)
print("\nNeighbours:")
for r in regions:
    print(f"  {r:>3} -> {neighbors[r]}")

solution = backtrack({})
if solution:
    print("\nSolution found:")
    for r in regions:
        print(f"  {r:>3} = {solution[r]}")
    ok = all(solution[r] != solution[n] for r in regions for n in neighbors[r])
    print("\nNo two adjacent regions share a colour:", ok)
```

```
else:
    print("\nNo solution exists")
```

Output:

Output - q1_mapcoloring.py

```
MAP COLORING PROBLEM (CSP - Backtracking)
Regions  : ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']
Colors   : ['Red', 'Green', 'Blue']

Neighbours:
   WA -> ['NT', 'SA']
   NT -> ['WA', 'SA', 'Q']
   SA -> ['WA', 'NT', 'Q', 'NSW', 'V']
    Q -> ['NT', 'SA', 'NSW']
  NSW -> ['Q', 'SA', 'V']
    V -> ['SA', 'NSW']
    T -> []

Solution found:
   WA = Red
   NT = Green
   SA = Blue
    Q = Red
  NSW = Green
    V = Red
    T = Red

No two adjacent regions share a colour: True
```

Result: The program for Map Coloring (CSP) was executed successfully and the output was verified.

Program 2: Tic Tac Toe Game

Course Outcome: CO3

Aim: To write a Python program to implement the Tic Tac Toe game in which a human player (X) plays against the computer (O).

Program:

```
WIN_LINES = [(0, 1, 2), (3, 4, 5), (6, 7, 8),
             (0, 3, 6), (1, 4, 7), (2, 5, 8),
             (0, 4, 8), (2, 4, 6)]


def print_board(b):
    for i in range(0, 9, 3):
        print(" " + " | ".join(b[i:i + 3]))
        if i < 6:
            print("---+---+---")
    print()


def winner(b):
    for x, y, z in WIN_LINES:
        if b[x] != " " and b[x] == b[y] == b[z]:
            return b[x]
    return None


def find_move(b, player):
    """Return a square that completes a line for `player`, else None."""
    for x, y, z in WIN_LINES:
        line = [b[x], b[y], b[z]]
        if line.count(player) == 2 and line.count(" ") == 1:
            return (x, y, z)[line.index(" ")]
    return None


def computer_move(b, me="O", opp="X"):
    move = find_move(b, me)             # 1. win
    if move is None:
        move = find_move(b, opp)        # 2. block
    if move is None and b[4] == " ":
        move = 4                        # 3. centre
    if move is None:
        for c in (0, 2, 6, 8, 1, 3, 5, 7):   # 4. corners, then edges
            if b[c] == " ":
                move = c
                break
    return move

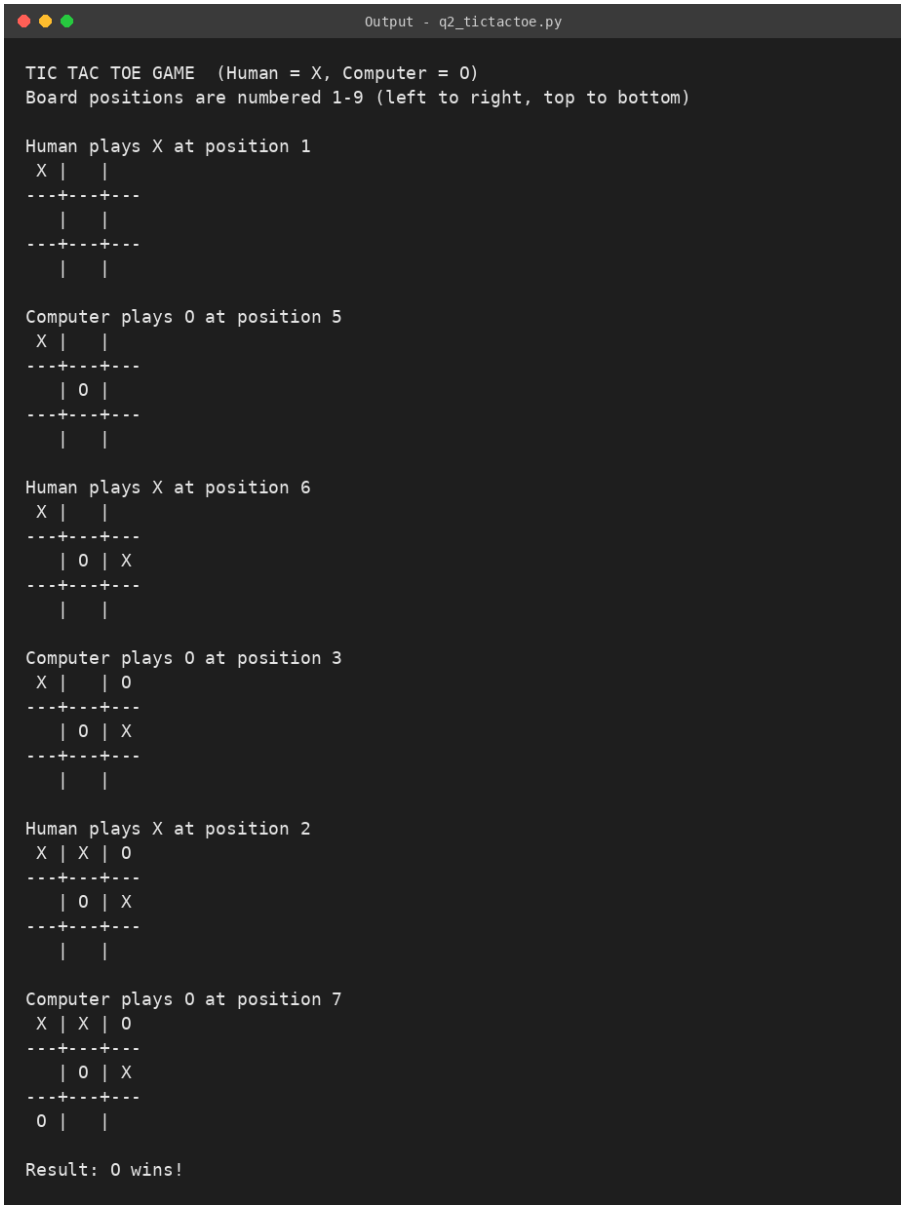

print("TIC TAC TOE GAME  (Human = X, Computer = O)")
print("Board positions are numbered 1-9 (left to right, top to bottom)\n")

board = [" "] * 9
```

```
human_moves = [1, 9, 6, 7]       # scripted human moves for demonstration
turn = "X"
for hm in human_moves + [None] * 5:
    if winner(board) or " " not in board:
        break
    if turn == "X":
        pos = hm - 1 if hm and board[hm - 1] == " " else board.index(" ")
        board[pos] = "X"
        print(f"Human plays X at position {pos + 1}")
        turn = "O"
    else:
        pos = computer_move(board)
        board[pos] = "O"
        print(f"Computer plays O at position {pos + 1}")
        turn = "X"
    print_board(board)

w = winner(board)
print("Result:", f"{w} wins!" if w else "It's a draw!")
```

Output:



Result: The program for Tic Tac Toe Game was executed successfully and the output was verified.

Program 3: Minimax Algorithm

Course Outcome: CO3

Aim: To write a Python program to implement the Minimax algorithm for two-player games and find the optimal value for the MAX player.

Program:

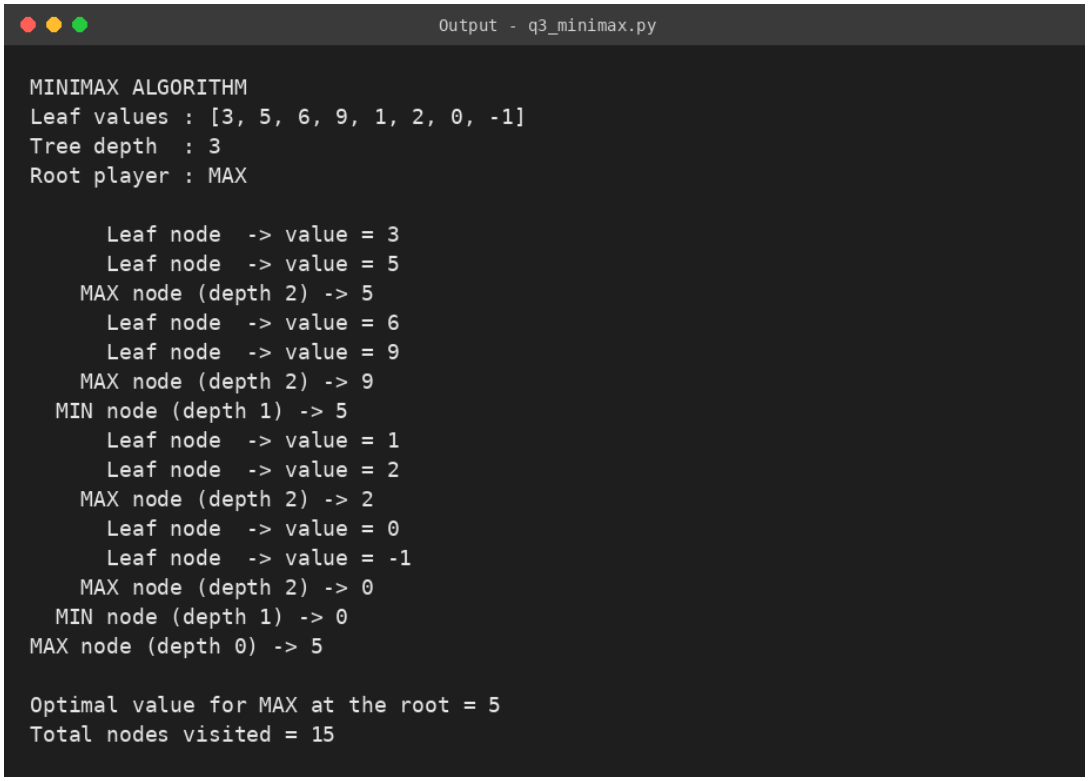
```
import math

# Leaf values of a complete binary game tree of depth 3
leaves = [3, 5, 6, 9, 1, 2, 0, -1]
nodes_visited = 0


def minimax(depth, index, is_max, values, max_depth):
    global nodes_visited
    nodes_visited += 1
    if depth == max_depth:
        print(f"{'  ' * depth}Leaf node  -> value = {values[index]}")
        return values[index]
    if is_max:
        best = -math.inf
        for child in (2 * index, 2 * index + 1):
            best = max(best, minimax(depth + 1, child, False, values, max_depth))
        print(f"{'  ' * depth}MAX node (depth {depth}) -> {best}")
        return best
    best = math.inf
    for child in (2 * index, 2 * index + 1):
        best = min(best, minimax(depth + 1, child, True, values, max_depth))
    print(f"{'  ' * depth}MIN node (depth {depth}) -> {best}")
    return best


depth = int(math.log2(len(leaves)))
print("MINIMAX ALGORITHM")
print("Leaf values :", leaves)
print("Tree depth  :", depth)
print("Root player : MAX\n")
result = minimax(0, 0, True, leaves, depth)
print("\nOptimal value for MAX at the root =", result)
print("Total nodes visited =", nodes_visited)
```

Output:



Result: The program for Minimax Algorithm was executed successfully and the output was verified.

Program 4: Alpha-Beta Pruning

Course Outcome: CO3

Aim: To write a Python program to implement the Alpha-Beta pruning algorithm for gaming, which gives the same result as Minimax while visiting fewer nodes.

Program:

```
import math

# Same game tree as the Minimax program
leaves = [3, 5, 6, 9, 1, 2, 0, -1]
nodes_visited = 0
pruned = 0


def alphabeta(depth, index, is_max, values, alpha, beta, max_depth):
    global nodes_visited, pruned
    nodes_visited += 1
    pad = "  " * depth
    if depth == max_depth:
        print(f"{pad}Leaf node -> value = {values[index]}")
        return values[index]
    if is_max:
        best = -math.inf
        for child in (2 * index, 2 * index + 1):
            val = alphabeta(depth + 1, child, False, values, alpha, beta, max_depth)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                print(f"{pad}PRUNED remaining branches of MAX node (alpha={alpha}, beta={beta})")
                pruned += 1
                break
        print(f"{pad}MAX node (depth {depth}) -> {best}")
        return best
    best = math.inf
    for child in (2 * index, 2 * index + 1):
        val = alphabeta(depth + 1, child, True, values, alpha, beta, max_depth)
        best = min(best, val)
        beta = min(beta, best)
        if beta <= alpha:
            print(f"{pad}PRUNED remaining branches of MIN node (alpha={alpha}, beta={beta})")
            pruned += 1
            break
    print(f"{pad}MIN node (depth {depth}) -> {best}")
    return best


depth = int(math.log2(len(leaves)))
print("ALPHA-BETA PRUNING ALGORITHM")
print("Leaf values :", leaves)
print("Tree depth  :", depth, "\n")
result = alphabeta(0, 0, True, leaves, -math.inf, math.inf, depth)
print("\nOptimal value for MAX at the root =", result)
print("Nodes visited   =", nodes_visited, "(Minimax visits 15)")
print("Pruning events  =", pruned)
```

Output:

Output - q4_alphabeta.py

```
ALPHA-BETA PRUNING ALGORITHM
Leaf values : [3, 5, 6, 9, 1, 2, 0, -1]
Tree depth  : 3

      Leaf node -> value = 3
      Leaf node -> value = 5
    MAX node (depth 2) -> 5
      Leaf node -> value = 6
    PRUNED remaining branches of MAX node (alpha=6, beta=5)
    MAX node (depth 2) -> 6
  MIN node (depth 1) -> 5
      Leaf node -> value = 1
      Leaf node -> value = 2
    MAX node (depth 2) -> 2
  PRUNED remaining branches of MIN node (alpha=5, beta=2)
  MIN node (depth 1) -> 2
MAX node (depth 0) -> 5

Optimal value for MAX at the root = 5
Nodes visited   = 11 (Minimax visits 15)
Pruning events  = 2
```

Result: The program for Alpha-Beta Pruning was executed successfully and the output was verified.

Program 5: Decision Tree

Course Outcome: CO4

Aim: To write a Python program to implement a Decision Tree classifier, train it on the Iris dataset and evaluate its accuracy.

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42)

model = DecisionTreeClassifier(criterion="entropy", max_depth=3, random_state=42)
model.fit(X_train, y_train)

print("DECISION TREE CLASSIFIER (Iris dataset)")
print("Training samples:", len(X_train), "| Testing samples:", len(X_test))
print("\nLearned tree rules:\n")
print(export_text(model, feature_names=list(iris.feature_names)))

pred = model.predict(X_test)
print("Accuracy on test data:", round(accuracy_score(y_test, pred) * 100, 2), "%")
print("\nConfusion matrix:")
print(confusion_matrix(y_test, pred))

sample = [[5.1, 3.5, 1.4, 0.2]]
print("\nPrediction for sample", sample[0], "->", iris.target_names[model.predict(sample)[0]])
```

Output:

Output - q5_decisiontree.py

```
DECISION TREE CLASSIFIER (Iris dataset)
Training samples: 105 | Testing samples: 45

Learned tree rules:

|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal length (cm) <= 4.75
|   |   |--- petal width (cm) <= 1.60
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.60
|   |   |   |--- class: 2
|   |--- petal length (cm) >  4.75
|   |   |--- petal length (cm) <= 5.15
|   |   |   |--- class: 2
|   |   |--- petal length (cm) >  5.15
|   |   |   |--- class: 2

Accuracy on test data: 97.78 %

Confusion matrix:
[[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]

Prediction for sample [5.1, 3.5, 1.4, 0.2] -> setosa
```

Result: The program for Decision Tree was executed successfully and the output was verified.

Program 6: Feed Forward Neural Network

Course Outcome: CO4

Aim: To write a Python program to implement a Feed Forward Neural Network trained with back-propagation to learn the XOR function.

Program:

```
import numpy as np

np.random.seed(1)


def sigmoid(x):
    return 1 / (1 + np.exp(-x))


def sigmoid_derivative(a):
    return a * (1 - a)


# XOR training data
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

# Network: 2 input -> 4 hidden -> 1 output
W1 = np.random.uniform(-1, 1, (2, 4))
b1 = np.zeros((1, 4))
W2 = np.random.uniform(-1, 1, (4, 1))
b2 = np.zeros((1, 1))
lr = 0.5

print("FEED FORWARD NEURAL NETWORK (XOR problem, back-propagation)")
print("Architecture: 2 input -> 4 hidden (sigmoid) -> 1 output (sigmoid)")
print("Learning rate:", lr, "\n")

for epoch in range(10001):
    # forward pass
    hidden = sigmoid(X @ W1 + b1)
    output = sigmoid(hidden @ W2 + b2)
    loss = np.mean((y - output) ** 2)
    # backward pass
    d_out = (output - y) * sigmoid_derivative(output)
    d_hid = (d_out @ W2.T) * sigmoid_derivative(hidden)
    W2 -= lr * hidden.T @ d_out
    b2 -= lr * d_out.sum(axis=0, keepdims=True)
    W1 -= lr * X.T @ d_hid
    b1 -= lr * d_hid.sum(axis=0, keepdims=True)
    if epoch % 2000 == 0:
        print(f"Epoch {epoch:5d} | Loss = {loss:.6f}")

print("\nFinal predictions:")
print("  Input   Expected   Output   Rounded")
for xi, yi, oi in zip(X, y, output):
    print(f"  {xi}     {yi[0]}        {oi[0]:.4f}     {int(round(oi[0]))}")
```

Output:

Output - q6_feedforward.py

```
FEED FORWARD NEURAL NETWORK (XOR problem, back-propagation)
Architecture: 2 input -> 4 hidden (sigmoid) -> 1 output (sigmoid)
Learning rate: 0.5

Epoch     0 | Loss = 0.249726
Epoch  2000 | Loss = 0.002766
Epoch  4000 | Loss = 0.000892
Epoch  6000 | Loss = 0.000516
Epoch  8000 | Loss = 0.000359
Epoch 10000 | Loss = 0.000275

Final predictions:
  Input   Expected   Output   Rounded
  [0 0]     0        0.0145     0
  [0 1]     1        0.9813     1
  [1 0]     1        0.9882     1
  [1 1]     0        0.0200     0
```

Result: The program for Feed Forward Neural Network was executed successfully and the output was verified.

Program 7: Sum of Integers from 1 to N (Prolog)

Course Outcome: CO4

Aim: To write a Prolog program to find the sum of the integers from 1 to N using recursion.

Program:

```
% Prolog program to find the sum of integers from 1 to N

sum(0, 0).
sum(N, S) :-
   N > 0,
   N1 is N - 1,
   sum(N1, S1),
   S is S1 + N.

main :-
   write('SUM OF INTEGERS FROM 1 TO N'), nl, nl,
   forall(member(N, [1, 5, 10, 100]),
        ( sum(N, S),
          format('?- sum(~w, S).~n   S = ~w~n', [N, S]) )),
   nl,
   format('Sum of 1 to 10 = 1+2+...+10 = ~w~n', [55]).
```

Output:

Output - q7_sum.py

```
SUM OF INTEGERS FROM 1 TO N

?- sum(1, S).
   S = 1
?- sum(5, S).
   S = 15
?- sum(10, S).
   S = 55
?- sum(100, S).
   S = 5050

Sum of 1 to 10 = 1+2+...+10 = 55
```

Result: The program for Sum of Integers from 1 to N (Prolog) was executed successfully and the output was verified.

Program 8: Database with Name and DOB (Prolog)

Course Outcome: CO5

Aim: To write a Prolog program to create a database of names and dates of birth and query it.

Program:

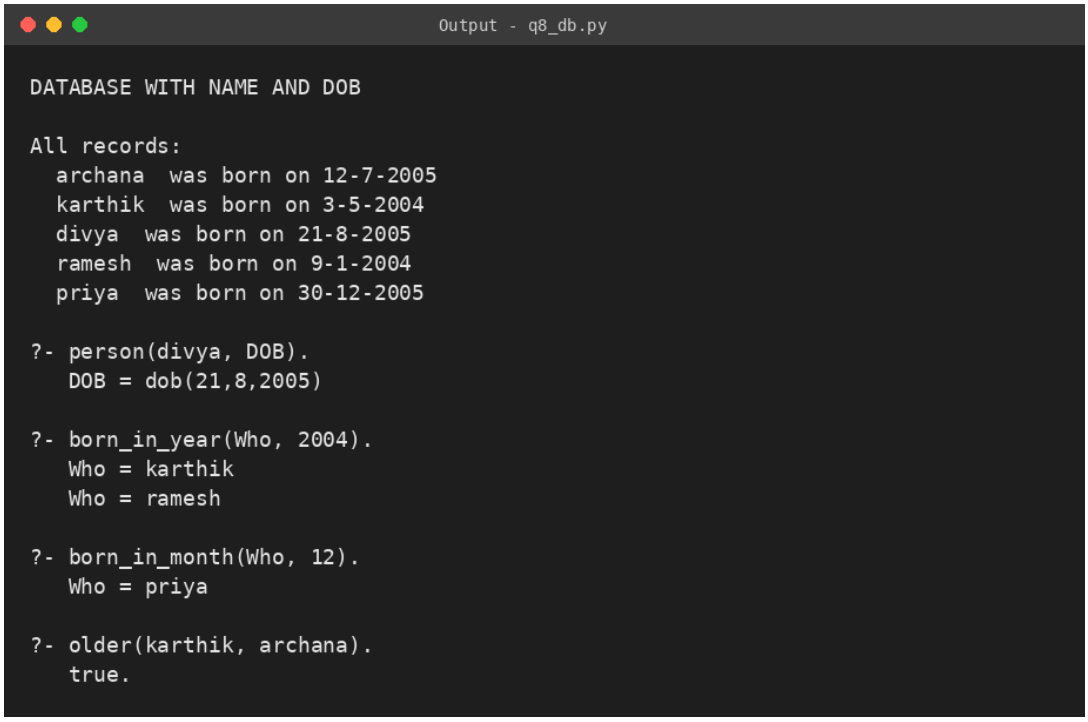
```
% Database of names and dates of birth: person(Name, dob(Day, Month, Year))

person(archana,  dob(12, 7, 2005)).
person(karthik,  dob(3, 5, 2004)).
person(divya,    dob(21, 8, 2005)).
person(ramesh,   dob(9, 1, 2004)).
person(priya,    dob(30, 12, 2005)).

born_in_year(Name, Year) :- person(Name, dob(_, _, Year)).
born_in_month(Name, Month) :- person(Name, dob(_, Month, _)).
older(A, B) :-
   person(A, dob(D1, M1, Y1)),
   person(B, dob(D2, M2, Y2)),
   A \= B,
   ( Y1 < Y2 ; Y1 =:= Y2, M1 < M2 ; Y1 =:= Y2, M1 =:= M2, D1 < D2 ).

main :-
   write('DATABASE WITH NAME AND DOB'), nl, nl,
   write('All records:'), nl,
   forall(person(N, dob(D, M, Y)),
        format('  ~w  was born on ~w-~w-~w~n', [N, D, M, Y])),
   nl,
   write('?- person(divya, DOB).'), nl,
   person(divya, DOB), format('   DOB = ~w~n', [DOB]), nl,
   write('?- born_in_year(Who, 2004).'), nl,
   forall(born_in_year(Who, 2004), format('   Who = ~w~n', [Who])), nl,
   write('?- born_in_month(Who, 12).'), nl,
   forall(born_in_month(Who2, 12), format('   Who = ~w~n', [Who2])), nl,
   write('?- older(karthik, archana).'), nl,
   ( older(karthik, archana) -> write('   true.') ; write('   false.') ), nl.
```

Output:



Result: The program for Database with Name and DOB (Prolog) was executed successfully and the output was verified.

Program 9: Student-Teacher-Subject Code (Prolog)

Course Outcome: CO1

Aim: To write a Prolog program to represent the relationship between students, teachers and subject codes and answer queries on it.

Program:

```
% STUDENT - TEACHER - SUBJECT CODE database

% teaches(Teacher, SubjectCode)
teaches(dr_kumar,  csa17).
teaches(dr_meena,  csa21).
teaches(mr_suresh, csa33).

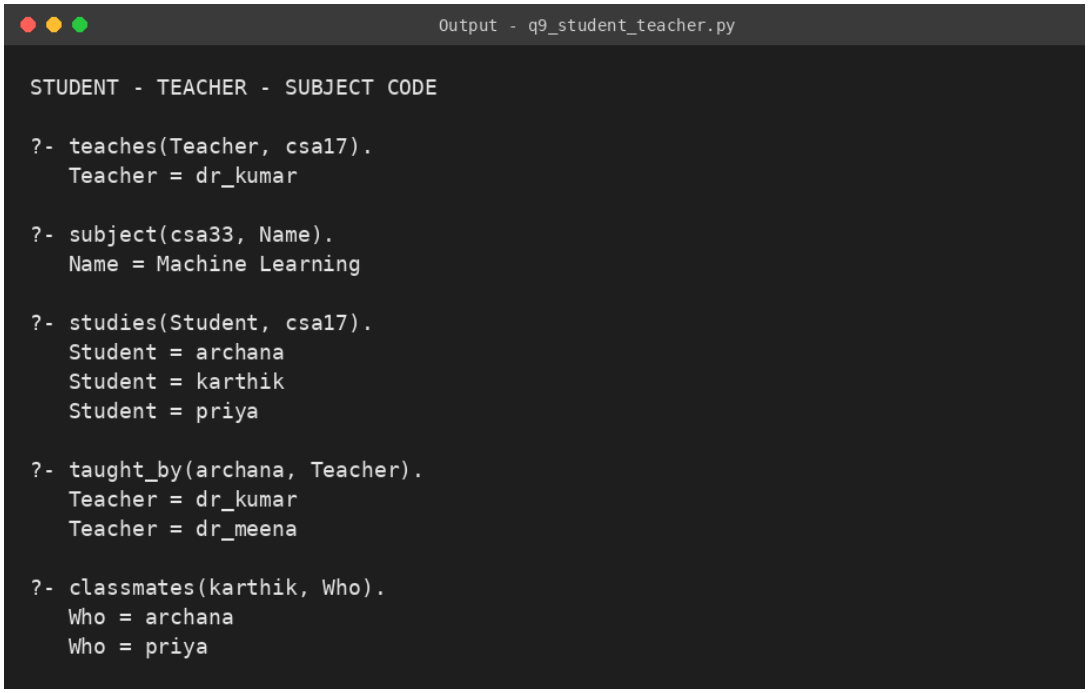
% subject(SubjectCode, SubjectName)
subject(csa17, 'Artificial Intelligence').
subject(csa21, 'Database Management Systems').
subject(csa33, 'Machine Learning').

% studies(Student, SubjectCode)
studies(archana, csa17).
studies(archana, csa21).
studies(karthik, csa17).
studies(divya,   csa33).
studies(ramesh,  csa21).
studies(priya,   csa17).

taught_by(Student, Teacher) :- studies(Student, Code), teaches(Teacher, Code).
classmates(A, B) :- studies(A, C), studies(B, C), A \= B.

main :-
    write('STUDENT - TEACHER - SUBJECT CODE'), nl, nl,
    write('?- teaches(Teacher, csa17).'), nl,
    forall(teaches(T, csa17), format('   Teacher = ~w~n', [T])), nl,
    write('?- subject(csa33, Name).'), nl,
    forall(subject(csa33, N), format('   Name = ~w~n', [N])), nl,
    write('?- studies(Student, csa17).'), nl,
    forall(studies(S, csa17), format('   Student = ~w~n', [S])), nl,
    write('?- taught_by(archana, Teacher).'), nl,
    forall(taught_by(archana, T2), format('   Teacher = ~w~n', [T2])), nl,
    write('?- classmates(karthik, Who).'), nl,
    forall(classmates(karthik, W), format('   Who = ~w~n', [W])).
```

Output:



Result: The program for Student-Teacher-Subject Code (Prolog) was executed successfully and the output was verified.

Program 10: Planets Database (Prolog)

Course Outcome: CO2

Aim: To write a Prolog program to create a database of planets and query their properties.

Program:

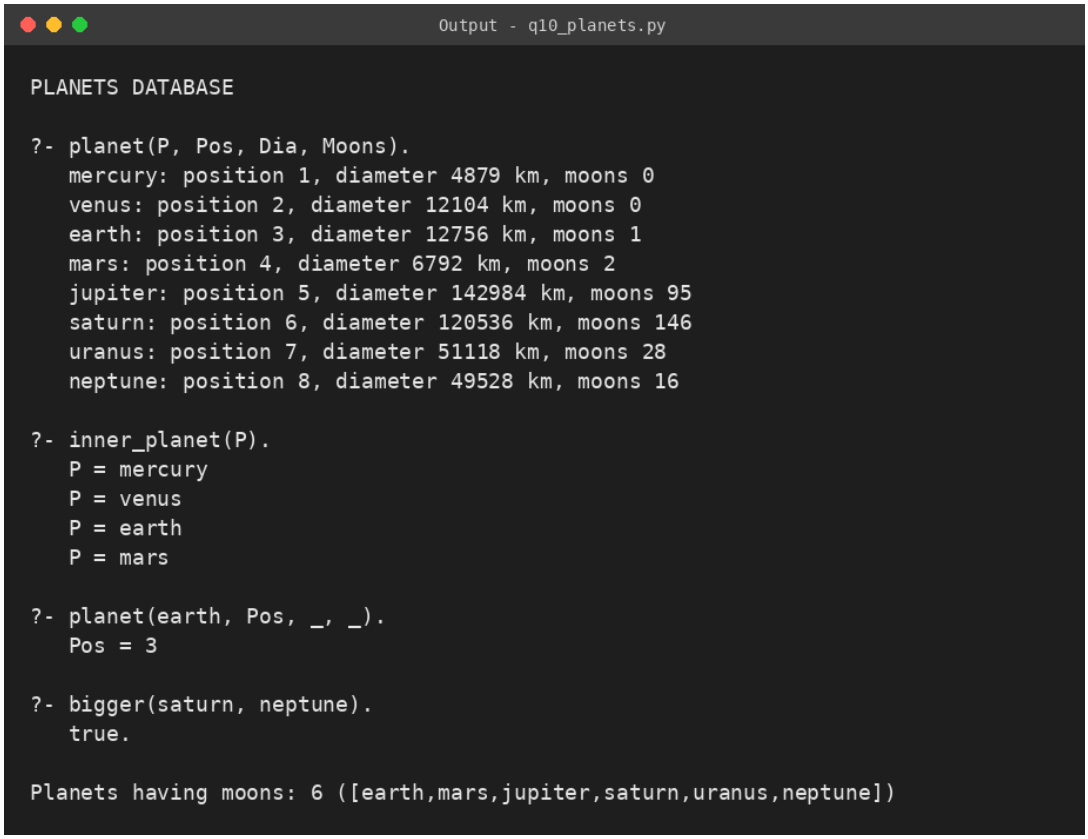
```
% PLANETS database: planet(Name, PositionFromSun, DiameterKm, Moons)

planet(mercury, 1,   4879, 0).
planet(venus,   2,  12104, 0).
planet(earth,   3,  12756, 1).
planet(mars,    4,   6792, 2).
planet(jupiter, 5, 142984, 95).
planet(saturn,  6, 120536, 146).
planet(uranus,  7,  51118, 28).
planet(neptune, 8,  49528, 16).

inner_planet(P)  :- planet(P, Pos, _, _), Pos =< 4.
outer_planet(P)  :- planet(P, Pos, _, _), Pos > 4.
has_moons(P)     :- planet(P, _, _, M), M > 0.
bigger(A, B)     :- planet(A, _, DA, _), planet(B, _, DB, _), DA > DB.

main :-
   write('PLANETS DATABASE'), nl, nl,
   write('?- planet(P, Pos, Dia, Moons).'), nl,
   forall(planet(P, Pos, Dia, Moons),
         format('   ~w: position ~w, diameter ~w km, moons ~w~n',
               [P, Pos, Dia, Moons])), nl,
   write('?- inner_planet(P).'), nl,
   forall(inner_planet(I), format('   P = ~w~n', [I])), nl,
   write('?- planet(earth, Pos, _, _).'), nl,
   planet(earth, EPos, _, _), format('   Pos = ~w~n', [EPos]), nl,
   write('?- bigger(saturn, neptune).'), nl,
   ( bigger(saturn, neptune) -> write('   true.') ; write('   false.') ), nl, nl,
   findall(M, has_moons(M), Ms), length(Ms, Count),
   format('Planets having moons: ~w (~w)~n', [Count, Ms]).
```

Output:



Result: The program for Planets Database (Prolog) was executed successfully and the output was verified.